Effects, made visible.

Start with explicit tracing

Describe meaningful host operations with lazy, nested spans.

```
system.span("load configuration", [],
  fn() { system.env("EYG_ORIGIN") },
  fn(_) { None },
)
```

Gleam owns trace state, OTLP encoding, batching, and errors. Plinthx binds only native APIs.

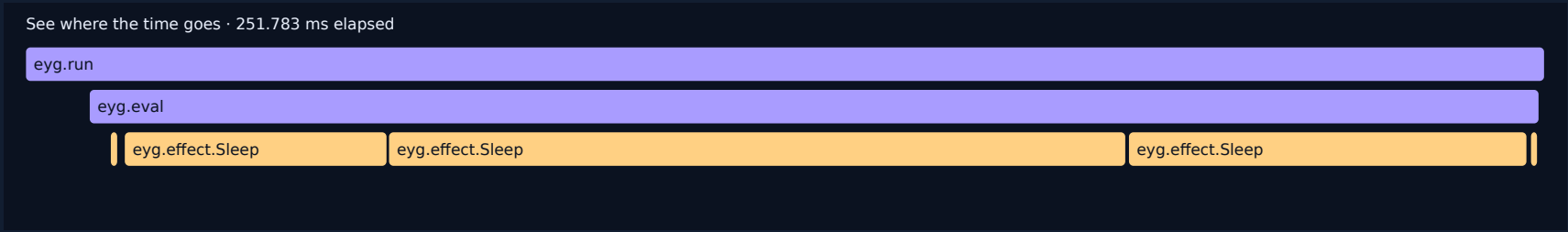
Then observe evaluation and effects

```
eyg.run
  eyg.eval
    eyg.effect.Fetch
    eyg.effect.Sleep
```

Parent width includes child work and waiting. These are instrumented wall-clock spans, not sampled CPU usage.

OTLP/HTTP → Jaeger or any compatible collector.

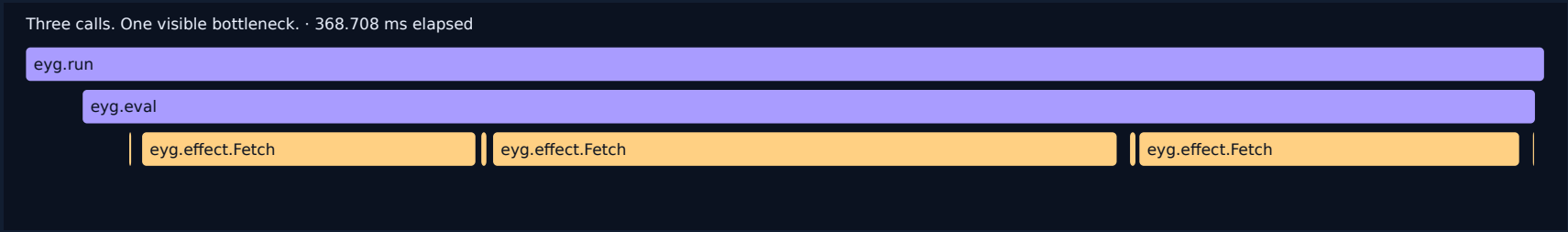
See where the time goes




```
// Each wait becomes a child span. Their widths reveal elapsed time.
// No tracing calls are needed in the program.
let _ = perform StandardOut("Starting three stages\n")
let _ = perform Sleep(40)
let _ = perform Sleep(120)
let _ = perform Sleep(65)
perform StandardOut("Three stages complete\n")
```

Exit 0 · trace 796e7aad506b3614a74634080fce20eb

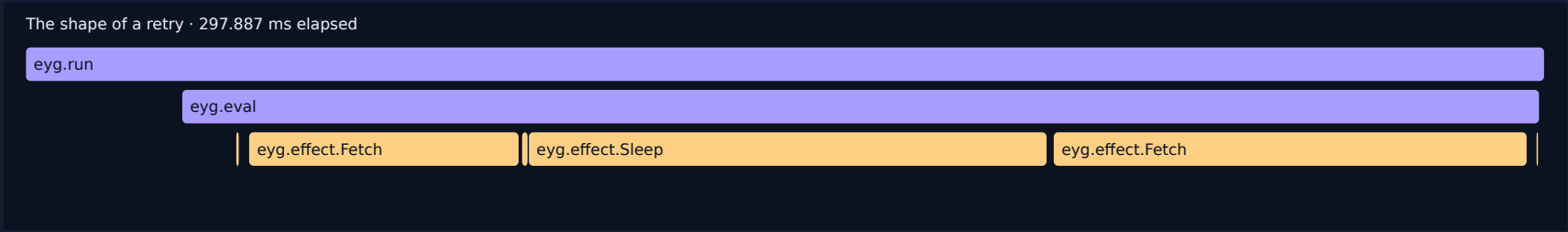
Three calls. One visible bottleneck.




```
// Local, deliberately delayed HTTP fixtures represent plan / tool / answer.
// These are real HTTP calls, not real model inference.
let http = import "./http.eyg"
let port = http.port({})
let plan = http.require_ok(http.get(port, "/plan"))
let _ = perform StandardOut(plan)
let tool = http.require_ok(http.get(port, "/tool"))
let _ = perform StandardOut(tool)
let answer = http.require_ok(http.get(port, "/answer"))
perform StandardOut(answer)
```

Exit 0 · trace 94e64f20c0eac1b5b21fd411683c6c7f

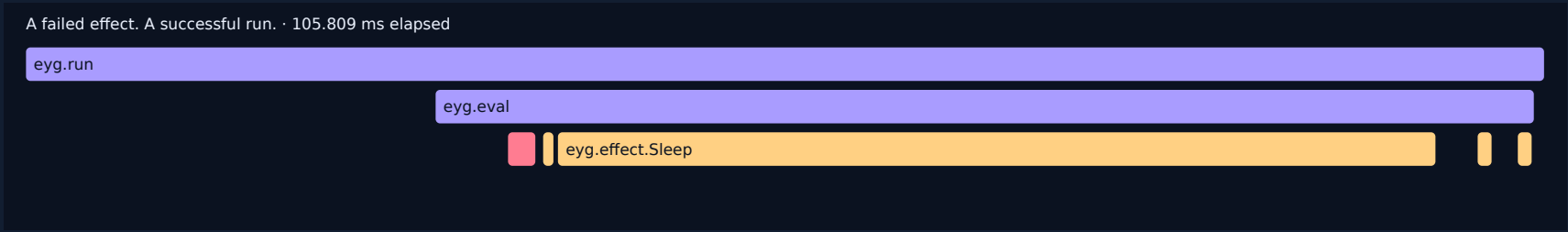
The shape of a retry




```
// An HTTP response is a successful Fetch, even when its status is 503.
// The application decides to retry; the backoff is visible as a Sleep span.
let http = import "./http.eyg"
let port = http.port({})
let first = http.get(port, "/unavailable")
match !equal(first.status, 503) {
  True(_) -> {
    let _ = perform StandardOut("HTTP 503; waiting before retry\n")
    let _ = perform Sleep(100)
    let answer = http.require_ok(http.get(port, "/answer"))
    perform StandardOut(answer)
  }
  False(_) -> { !never(perform Abort("expected the demo's HTTP 503")) }
}
```

Exit 0 · trace b1ee08dee64b3c631dc329222352f0ba

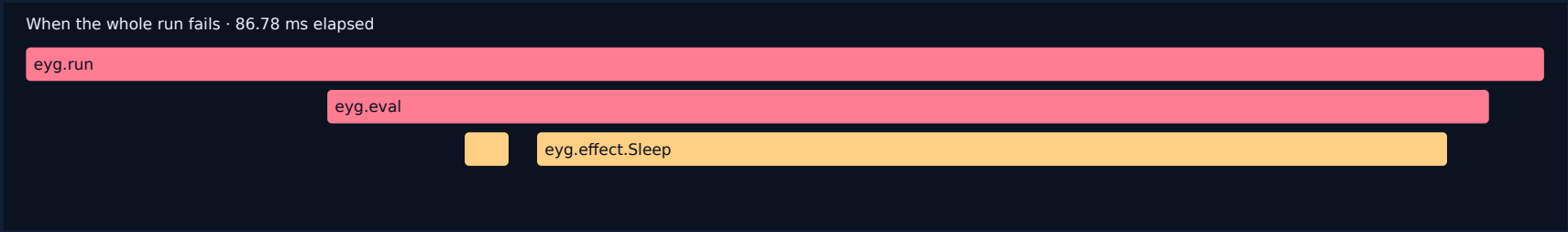
A failed effect. A successful run.




```
// Reading a directory as a file fails deterministically.
// The effect span is red, but handling its Error keeps the run successful.
let text = match perform ReadFile({path: "./fixtures", offset: 0, limit: 1024}) {
  Ok(_) -> { !never(perform Abort("expected the directory read to fail")) }
  Error(_) -> {
    let _ = perform StandardOut("Read failed; using the fallback\n")
    let _ = perform Sleep(60)
    match perform ReadFile({path: "./fixtures/fallback.txt", offset: 0, limit:
1024}) {
      Ok(bytes) -> {
        match !string_from_binary(bytes) {
          Ok(text) -> { text }
          Error(_) -> { !never(perform Abort("fallback was not UTF-8")) }
        }
      }
      Error(reason) -> { !never(perform Abort(reason)) }
    }
  }
}
perform StandardOut(text)
```

Exit 0 · trace 21a68dd468d1f59806769c6951f25db4

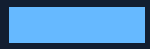
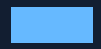
When the whole run fails




```
// Abort terminates evaluation. The root and evaluation spans become errors.
// Abort itself has no computer-harness implementation, so it has no effect span.
let _ = perform StandardOut("Beginning a task that will fail\n")
let _ = perform Sleep(50)
perform Abort("demo: task could not be completed")
```

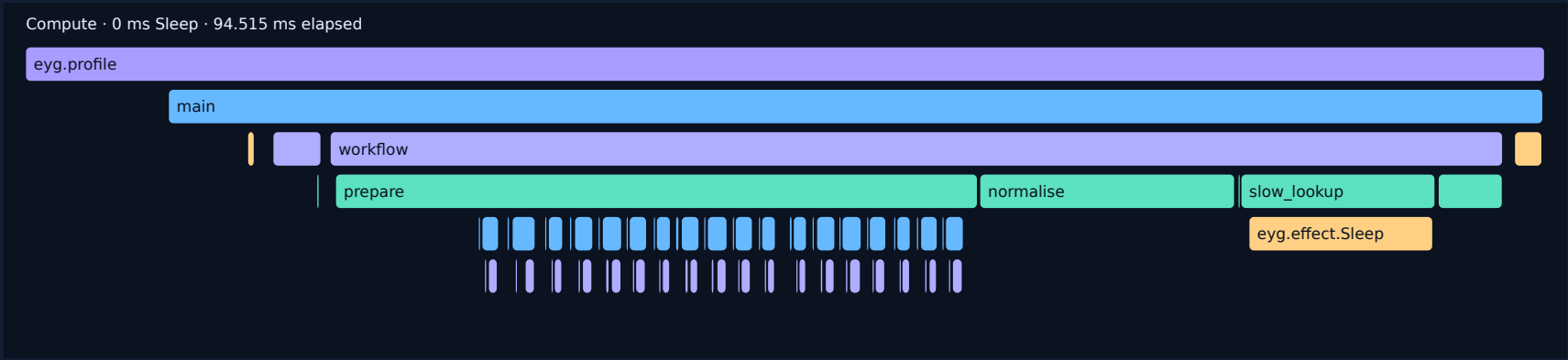
Exit 1 · trace 02d3688fd553b96d8e2524a2f259eda5

Read the HTTP workflow as a timeline

Effect	Start ms	Duration ms	Timeline
eyg.effect.Env	25.072	0.462	
Fetch /plan → 200	28.201	80.951	
eyg.effect.StandardOut	110.586	1.238	
Fetch /tool → 200	113.448	151.453	
eyg.effect.StandardOut	268.149	1.322	
Fetch /answer → 200	270.429	92.229	
eyg.effect.StandardOut	366.015	0.213	

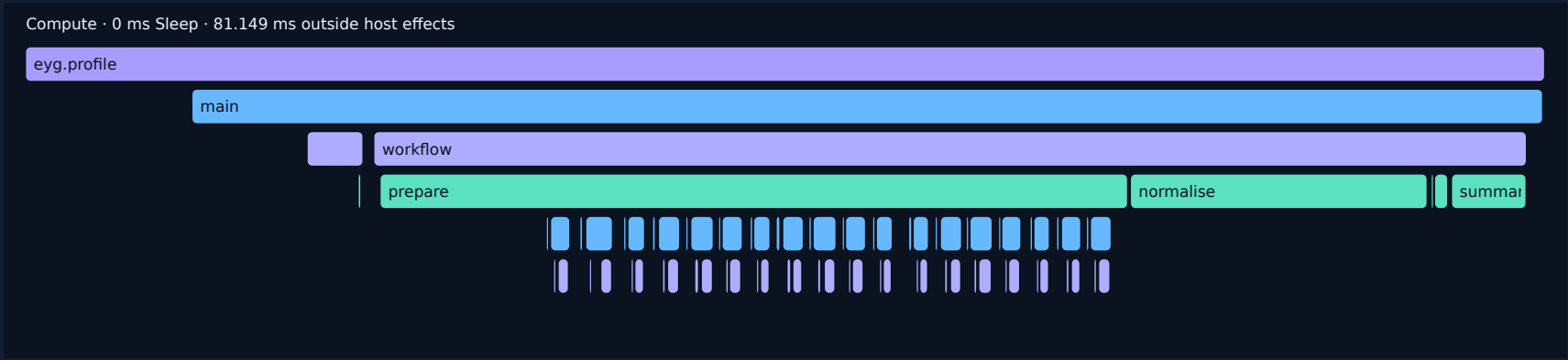
Routes come from the fixture server's request log. Delays are deliberate local fixtures, not model benchmarks.

Compute · 0 ms Sleep · elapsed



Curried applications and callbacks retain definition names. Continuation re-entry records new segments with the same logical call ID.

Compute · 0 ms Sleep · evaluation



Host-effect intervals are removed once. Interpreter, GC, scheduling, and instrumentation overhead remain.

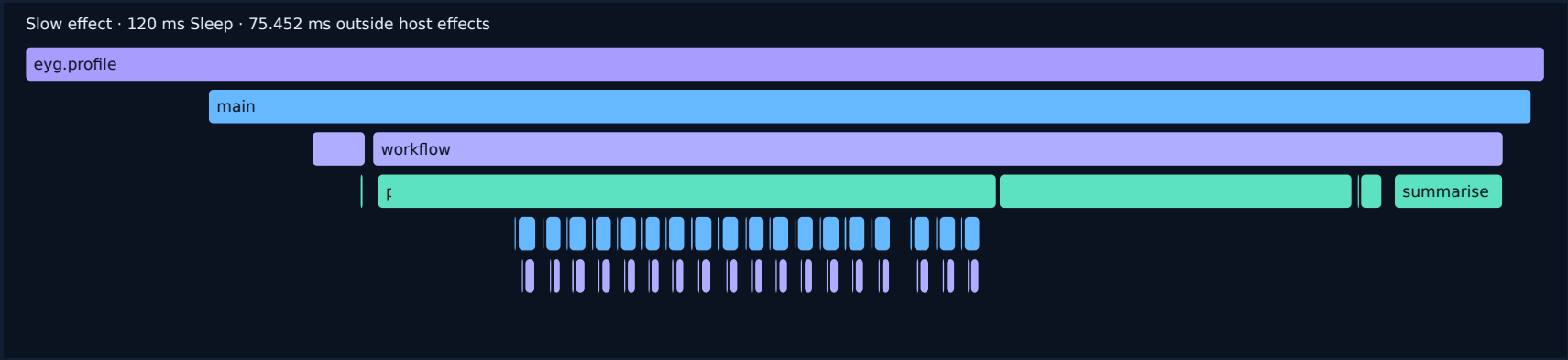
Open interactive performance view

Slow effect · 120 ms Sleep · elapsed



Curried applications and callbacks retain definition names. Continuation re-entry records new segments with the same logical call ID.

Slow effect · 120 ms Sleep · evaluation



Host-effect intervals are removed once. Interpreter, GC, scheduling, and instrumentation overhead remain.

Open interactive performance view

Errors describe execution

A failed ReadFile can be recovered by EYG. An unhandled Abort fails the evaluation. A returned HTTP 503 is still a successful Fetch transport response.

Execution evidence is not a task-quality score, and trace storage is not a replay log.

Run your own recording

```
# From packages/tracing
gleam run -- capture
gleam run -- profile-capture
gleam run -- export
```

Performance view · Jaeger · Tracing guide